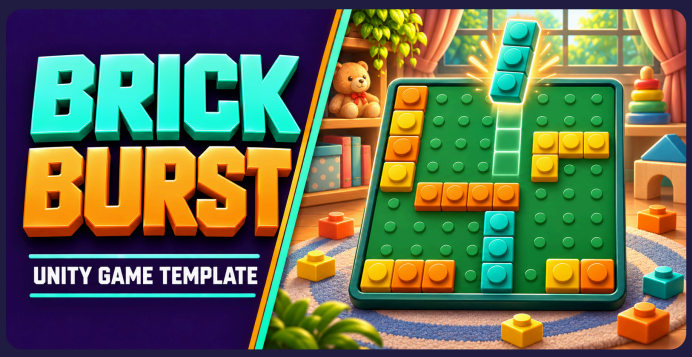
UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

# Brick Burst Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

# Start here: Brick Burst

Brick Burst is a standalone editable Unity game template. Drag one of three tray bricks onto free cells of the 8 by 8 baseplate. Full rows and columns clear. Meet the visible score or prize goal while retaining legal placements. Popper, steel and gift bricks alter the board. Free Play is a separate local score run in Settings.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

# Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Brick Burst > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/BlastBlocks/Scenes/BlastBlocks_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

# Project folders and exact source map

All game-relative paths below start at `Assets/Games/BlastBlocks`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/blastblocks contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/BbArt.cs`
- `Scripts/BbData.cs`
- `Scripts/BbGen.cs`
- `Scripts/BbLevels.cs`
- `Scripts/BbLogic.cs`
- `Scripts/BbSession.cs`
- `Scripts/BbSfx.cs`
- `Scripts/BlastBlocksGame.cs`
- `Scripts/BlastBlocksGame.Hud.cs`
- `Scripts/BlastBlocksGame.View.cs`

Main flow: BlastBlocksGame.cs. UI and popup consumers: BlastBlocksGame.Hud.cs. Model/generator/loading: BbData.cs, BbGen.cs, BbLevels.cs, BbLogic.cs. Authoring: Editor/BlastBlocksLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

# Gameplay, controls and progression

Drag one of three tray bricks onto free cells of the 8 by 8 baseplate. Full rows and columns clear. Meet the visible score or prize goal while retaining legal placements. Popper, steel and gift bricks alter the board. Free Play is a separate local score run in Settings.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

# Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/BlastBlocksGame.cs; effect/help strings: Scripts/BlastBlocksGame.cs.

| Helper | Unlock | Free charges | Action and effect |
|---|---|---|---|
| Reroll | 3 | 3 | Swap all the bricks in your tray for new ones. |
| Pocket | 5 | 3 | Drag a brick into the Pocket to keep it for later. Drag it out when you need it. |
| Rocket | 8 | 2 | Tap Rocket, then tap a row: the whole row blasts away, steel too! |
| Hammer | 11 | 2 | Tap Hammer, then tap any brick to smash it, steel too! |

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

# Visual level editor: worked editing session

Open **Szekipapa77 > Brick Burst > Level Editor**. Exact implementation: `Editor/BlastBlocksLevelEditor.cs`.

Open the Brick Burst editor and select an early level. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/blastblocks/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

| Source | Exact current code |
| --- | --- |
| `BlastBlocksLevelEditor.cs:39` | `[MenuItem("Szekipapa77/Brick Burst/Level Editor")]` |
| `BlastBlocksLevelEditor.cs:42` | `[MenuItem("Szekipapa77/Brick Burst/Validate All Levels")]` |
| `BlastBlocksLevelEditor.cs:89` | `if (GUILayout.Button("Save", EditorStyles.toolbarButton, GUILayout.Width(60))) Save();` |
| `BlastBlocksLevelEditor.cs:90` | `if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!Dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();` |
| `BlastBlocksLevelEditor.cs:91` | `if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(90))) ValidateAll();` |

| | |
|---|---|
| `BlastBlocksLevelEditor.cs:114` | `if (GUILayout.Button("Add")) { Record("Add level"); var l = new List<BbLevelData>(_file.levels); var d = new BbLevelData { seed = UnityEngine.Random.Range(1, 99999) }; d.Normalize(); l.Insert(Mathf.Min(_sel + 1, l.Count), d); _file.levels = l.ToArray(); _sel = Mathf.Min(_sel + 1, l.Count - 1); _results.Clear(); Dirty = true; }` |
| `BlastBlocksLevelEditor.cs:116` | `if (GUILayout.Button("Dup")) { Record("Duplicate level"); var l = new List<BbLevelData>(_file.levels); var d = Cur.Clone(); d.name += " copy"; l.Insert(_sel + 1, d); _file.levels = l.ToArray(); _sel++; _results.Clear(); Dirty = true; }` |
| `BlastBlocksLevelEditor.cs:117` | `if (GUILayout.Button("Del") && EditorUtility.DisplayDialog("Delete level", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel")) { Record("Delete level"); var l = new List<BbLevelData>(_file.levels); l.RemoveAt(_sel); _file.levels = l.ToArray(); _sel = Mathf.Clamp(_sel, 0, l.Count - 1); _results.Clear(); Dirty = true; }` |
| `BlastBlocksLevelEditor.cs:118` | `if (GUILayout.Button("Up") && _sel > 0) { Record("Move level"); (_file.levels[_sel], _file.levels[_sel - 1]) = (_file.levels[_sel - 1], _file.levels[_sel]); _sel--; _results.Clear(); Dirty = true; }` |
| `BlastBlocksLevelEditor.cs:119` | `if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) { Record("Move level"); (_file.levels[_sel], _file.levels[_sel + 1]) = (_file.levels[_sel + 1], _file.levels[_sel]); _sel++; _results.Clear(); Dirty = true; }` |
| `BlastBlocksLevelEditor.cs:146` | `if (d == null) { EditorGUILayout.HelpBox("No levels. Press Add or Auto Generate.", MessageType.Info); return; }` |
| `BlastBlocksLevelEditor.cs:147` | `EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.largeLabel);` |
| `BlastBlocksLevelEditor.cs:153` | `string name = EditorGUILayout.TextField("Name", d.name);` |
| `BlastBlocksLevelEditor.cs:154` | `string hint = EditorGUILayout.TextField("First-time hint", d.hint);` |

| | |
|---|---|
| `BlastBlocksLevelEditor.cs:156` | `g = EditorGUILayout.Popup("Goal", g, GoalNames);` |
| `BlastBlocksLevelEditor.cs:158` | `if (g == 0) target = EditorGUILayout.IntSlider("Score target", d.target, 20, 2000);` |
| `BlastBlocksLevelEditor.cs:161` | `stars = EditorGUILayout.IntSlider("Stars to collect", d.stars, 0, 40);` |
| `BlastBlocksLevelEditor.cs:162` | `hearts = EditorGUILayout.IntSlider("Hearts to collect", d.hearts, 0, 40);` |
| `BlastBlocksLevelEditor.cs:163` | `ducks = EditorGUILayout.IntSlider("Ducks to collect", d.ducks, 0, 40);` |
| `BlastBlocksLevelEditor.cs:166` | `pool = EditorGUILayout.Popup("Piece pool", pool, Dealer.Pools);` |
| `BlastBlocksLevelEditor.cs:167` | `string opening = EditorGUILayout.TextField(new GUIContent("Opening tray", "Fixed first pieces, e.g. I4.0 I3.1 O2.0"), d.opening);` |
| `BlastBlocksLevelEditor.cs:168` | `int seed = EditorGUILayout.IntField("Seed", d.seed);` |
| `BlastBlocksLevelEditor.cs:169` | `float diff = EditorGUILayout.Slider("Design difficulty", d.difficulty, 0, 1);` |
| `BlastBlocksLevelEditor.cs:170` | `float help = EditorGUILayout.Slider(new GUIContent("Help", "Chance a tray offers a line clear"), d.help, 0, 1);` |
| `BlastBlocksLevelEditor.cs:171` | `float pop = EditorGUILayout.Slider(new GUIContent("Popper rate", "Chance a dealt piece carries a popper"), d.popperRate, 0, 0.4f);` |
| `BlastBlocksLevelEditor.cs:172` | `float prize = EditorGUILayout.Slider(new GUIContent("Prize rate", "Chance a dealt piece carries an owed prize"), d.prizeRate, 0.1f, 1);` |
| `BlastBlocksLevelEditor.cs:173` | `bool breather = EditorGUILayout.Toggle("Breather", d.breather);` |
| `BlastBlocksLevelEditor.cs:174` | `int par = EditorGUILayout.IntField(new GUIContent("Par (3 stars)", "Pieces for 3 stars; set by Validate"), d.par);` |
| `BlastBlocksLevelEditor.cs:186` | `if (GUILayout.Button("Validate", GUILayout.Height(28))) ValidateOne(_sel);` |

| | |
|---|---|
| `BlastBlocksLevelEditor.cs:187` | `if (GUILayout.Button("Play this level", GUILayout.Height(28))) Play(_sel + 1);` |
| `BlastBlocksLevelEditor.cs:190` | `EditorGUILayout.HelpBox(v.ToString() + (v.Solvable ? $"\npar set to {d.par} pieces (3 stars)" : ""), v.Solvable ? MessageType.Info : MessageType.Error);` |
| `BlastBlocksLevelEditor.cs:191` | `else if (d.score >= 0) EditorGUILayout.HelpBox($"Last validation: bot {d.botPieces:0} pieces, casual {d.botRate * 100:0}%, difficulty {d.score:0}", MessageType.None);` |
| `BlastBlocksLevelEditor.cs:196` | `EditorGUILayout.LabelField("Left-click/drag paints, right-click erases.", EditorStyles.miniLabel);` |
| `BlastBlocksLevelEditor.cs:201` | `if (GUILayout.Button("Apply pattern"))` |
| `BlastBlocksLevelEditor.cs:208` | `if (GUILayout.Button("Clear board")) { Record("Clear board"); for (int i = 0; i < Board.N; i++) d.Set(i / Board.Size, i % Board.Size, '.'); Changed(); }` |
| `BlastBlocksLevelEditor.cs:214` | `if (full > 0) EditorGUILayout.HelpBox($"{full} line(s) start full: they will never clear by themselves. Leave a gap.", MessageType.Warning);` |
| `BlastBlocksLevelEditor.cs:215` | `if (d.Goal == GoalKind.Collect && d.PrizeTotal == 0) EditorGUILayout.HelpBox("Collect goal with nothing to collect.", MessageType.Warning);` |
| `BlastBlocksLevelEditor.cs:257` | `EditorGUILayout.LabelField("Difficulty curve (bot score per level; blue = breather, red = unsolved). Click a bar to select.", EditorStyles.boldLabel);` |
| `BlastBlocksLevelEditor.cs:275` | `EditorGUILayout.LabelField("Auto Generate (the same BbGen code as the endless levels; every level is bot-validated)", EditorStyles.boldLabel);` |
| `BlastBlocksLevelEditor.cs:278` | `_genCount = EditorGUILayout.IntField("Count", _genCount); _genStart = EditorGUILayout.IntField("Start level", _genStart); _genSeed = EditorGUILayout.IntField("Seed", _genSeed);` |

| | |
|---|---|
| `BlastBlocksLevelEditor.cs:282` | `_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);` |
| `BlastBlocksLevelEditor.cs:286` | `_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 3, 8);` |
| `BlastBlocksLevelEditor.cs:287` | `_genReplace = EditorGUILayout.ToggleLeft("Replace range (else insert)", _genReplace);` |
| `BlastBlocksLevelEditor.cs:299` | `if (GUILayout.Button($"Generate {_genCount} levels at {_genStart}", GUILayout.Height(28))) Generate();` |

CHAPTER 07

# Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

| Source | Exact current code |
| --- | --- |
| `BlastBlocksLevelEditor.cs:37` | `bool _genReplace;` |
| `BlastBlocksLevelEditor.cs:287` | `_genReplace = EditorGUILayout.ToggleLeft("Replace range (else insert)", _genReplace);` |
| `BlastBlocksLevelEditor.cs:325` | `if (_genReplace && at + k < list.Count) list[at + k] = d; else list.Insert(Mathf.Min(at + k, list.Count), d);` |

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

# Actual level JSON and identifiers

Canonical asset: `Assets/Games/BlastBlocks/Resources/blastblocks/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "name": "First Bricks",
  "pattern": "tutorial",
  "hint": "Drag a brick into the gap to fill a whole row. Full rows and columns go POP!",
  "grid": [
    "b.......",
    "b.......",
    "b.......",
    "b.......",
    "b.......",
    "........",
    "........",
    "bb....bb"
  ],
  "goal": "score",
  "target": 150,
  "stars": 0,
  "hearts": 0,
  "ducks": 0,
  "pool": "easy",
  "opening": "I4.0 I3.1 O2.0",
  "seed": 1013,
  "difficulty": 0.05,
  "help": 0.7,
  "popperRate": 0,
  "prizeRate": 0.6,
  "breather": false,
  "par": 10,
  "botRate": 0.6666667,
  "botPieces": 6,
  "score": 25
}
```


Root keys: levels. Inspect BbData.cs, BbGen.cs, BbLevels.cs, BbLogic.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

# Reskin recipe: interface colours

Stop Play mode and open `Scripts/BbArt.cs`. Find `Ink`, currently `#22305A`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

| Source | Exact current code |
| --- | --- |
| `BlastBlocksGame.Hud.cs:73 / void Pill(float x, float y, float w, float h)` | `var pill = HzUI.Panel("LevelPill", _hudRoot, BbArt.Ink, (int)(h / 2)); pill.raycastTarget = false;` |
| `BlastBlocksGame.Hud.cs:82 / void SettingsButton(float x, float y, float size)` | `var b = HzIcons.IconButton(_hudRoot, HzIcons.Settings, BbArt.Ink, () => { if (!PopupOpen) ShowSettingsPopup(); }, size);` |
| `BlastBlocksGame.Hud.cs:91 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `var rim = HzUI.Img("Rim", card.transform, HzUI.RoundRect(34, 5), BbArt.Ink); rim.type = Image.Type.Sliced; rim.preserveAspect = false; rim.raycastTarget = false; rim.rectTransform.Stretch();` |
| `BlastBlocksGame.Hud.cs:97 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_scoreText = HzUI.Label("Score", ct, "0", 76, BbArt.Ink, TextAnchor.MiddleLeft);` |
| `BlastBlocksGame.Hud.cs:105 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_comboText.AddOutline(BbArt.Ink, 4);` |
| `BlastBlocksGame.Hud.cs:112 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_scoreText = HzUI.Label("Score", ct, "0", 96, BbArt.Ink);` |
| `BlastBlocksGame.Hud.cs:120 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_comboText.AddOutline(BbArt.Ink, 4);` |
| `BlastBlocksGame.Hud.cs:132 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `var cnt = HzUI.Label("Count", slot, "0", portrait ? 52 : 56, BbArt.Ink, TextAnchor.MiddleLeft);` |
| `BlastBlocksGame.Hud.cs:173 / void BoosterButton(int b, float x, float y)` | `var lbl = HzUI.Label("Name", _hudRoot, BoostName[b], 34, BbArt.Ink);` |

| | |
|---|---|
| BlastBlocksGame.Hud.cs:329 / void WinPopup(int stars) | st.AddOutline(BbArt.Ink, 4); |
| BlastBlocksGame.Hud.cs:422 / void ShowHand(Vector2 from, Vector2 to, bool drag) | l.AddOutline(BbArt.Ink, 5); l.AddShadow(new Color(0, 0, 0, 0.35f), 8); |
| BlastBlocksGame.View.cs:124 / void BuildView() | var frame = HzUI.Panel("Frame", _boardRoot, BbArt.Ink, 40); |
| BlastBlocksGame.View.cs:158 / void BuildView() | var o = HzUI.Img("Rim", sb.transform, HzUI.RoundRect(34, 4), new Color(BbArt.Ink.r, BbArt.Ink.g, BbArt.Ink.b, 0.25f)); o.type = Image.Type.Sliced; o.preserveAspect = false; o.raycastTarget = false; o.rectTransform.Stretch(); |
| BlastBlocksGame.View.cs:608 / void FloatText(string text, Vector2 at, Color color, int size, float dur) | l.AddOutline(BbArt.Ink, 4); |
| BlastBlocksGame.View.cs:624 / void Praise(string text, bool big = false) | l.AddOutline(BbArt.Ink, 6); l.AddShadow(new Color(0, 0, 0, 0.35f), 6); |

| Source | Exact current code |
|---|---|
| BbArt.cs:9 / class/static declarations | public static readonly Color Ink = HzUI.Hex("#22305A"), InkSoft = HzUI.Hex("#3C4D80"), Cream = HzUI.Hex("#FFF8EA"), Plate = HzUI.Hex("#2FAE5A"), |
| BbArt.cs:10 / class/static declarations | PlateDark = HzUI.Hex("#1F8A45"), StudColor = HzUI.Hex("#45C775"), SkyTop = HzUI.Hex("#7FD8FF"), SkyBot = HzUI.Hex("#C9F5E4"), |
| BbArt.cs:11 / class/static declarations | Gold = HzUI.Hex("#FFC93C"), Good = HzUI.Hex("#3DBE6E"), Bad = HzUI.Hex("#FF4D5E"), Orange = HzUI.Hex("#FF9F1C"), Blue = HzUI.Hex("#3A86FF"), |
| BbArt.cs:12 / class/static declarations | Purple = HzUI.Hex("#9B5DE5"), Pink = HzUI.Hex("#FF6FB5"), Teal = HzUI.Hex("#1FC8B5"), Locked = HzUI.Hex("#8C97B8"), Steel = HzUI.Hex("#9AA9C2"), |
| BbArt.cs:13 / class/static declarations | Card = HzUI.Hex("#FFFFFF"), Sub = HzUI.Hex("#6B7AA6"); |

| | |
|---|---|
| `BbArt.cs:18 / class/static declarations` | `Color.white, HzUI.Hex("#FF4D5E"), HzUI.Hex("#FF9F1C"), HzUI.Hex("#FFD60A"), HzUI.Hex("#7BD639"), HzUI.Hex("#1FC8B5"), HzUI.Hex("#3A86FF"), HzUI.Hex("#9B5DE5"), HzUI.Hex("#FF6FB5"), HzUI.Hex("#B5C2D8"),` |
| `BbArt.cs:21 / class/static declarations` | `public static readonly Color[] PrizeColor = { HzUI.Hex("#FFC93C"), HzUI.Hex("#FF6FB5"), HzUI.Hex("#FFD60A") };` |
| `BlastBlocksGame.cs:35 / class/static declarations` | `static readonly Color[] BoostColor = { HzUI.Hex("#FF6B6B"), HzUI.Hex("#1FC8B5"), HzUI.Hex("#3A86FF"), HzUI.Hex("#FF9F1C") };` |

CHAPTER 10

# Reskin recipe: visible menu and HUD text

Open `Scripts/BlastBlocksGame.Hud.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/BlastBlocksGame.cs; effect/help strings: Scripts/BlastBlocksGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

| Source | Exact current code |
| --- | --- |
| `BbArt.cs:24 / class/static declarations` | `public static readonly string[] PrizeName = { "Stars", "Hearts", "Ducks" };` |
| `BbSession.cs:22 / class/static declarations` | `Difficulty = d.difficulty; Help = d.help; PopperRate = d.popperRate; PrizeRate = d.prizeRate; Pool = d.pool ?? "mixed";` |
| `BlastBlocksGame.cs:25 / class/static declarations` | `static readonly string[] BoostName = { "Reroll", "Pocket", "Rocket", "Hammer" };` |
| `BlastBlocksGame.cs:79 / void StartEndless()` | `Toast("Free Play: score as much as you can!", null, 2f, 0.82f);` |
| `BlastBlocksGame.cs:178 / void OutOfSpace()` | `var p = Popup("Out of Space!", "No brick fits anymore.\n" + ProgressText(), BbArt.Bad, buttons.ToArray());` |
| `BlastBlocksGame.cs:201 / void Rescue()` | `Toast("Space cleared!", null, 1.2f, 0.8f);` |
| `BlastBlocksGame.cs:211 / void EndlessOver()` | `var p = Popup(record ? "New Best!" : "Game Over", $"Score {S.Score}\nBest {Best}", record ? BbArt.Gold : BbArt.Bad,` |
| `BlastBlocksGame.cs:309 / void OnBooster(int b)` | `if (!Unlocked(b)) { HzAudio.Play(Sfx.Invalid, 0.5f); Toast($"{BoostName[b]} unlocks at level {BoostUnlock[b]}", null, 1.4f, 0.3f); return; }` |

| | |
|---|---|
| `BlastBlocksGame.cs:352 / void OfferCharge(int b)` | `var p = Popup($"More {BoostName[b]}?", "Watch a short ad to get one more charge.", BoostColor[b],` |
| `BlastBlocksGame.cs:353 / void OfferCharge(int b)` | `("Get +1 (Ad)", BbArt.Good, (Action)(() => HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant(), ok =>` |
| `BlastBlocksGame.cs:401 / void TryStorePocket(int t)` | `if (S.Pocket != null) { HzAudio.Play(Sfx.Invalid, 0.5f); Toast("The Pocket is full", null, 1.2f, 0.3f); return; }` |
| `BlastBlocksGame.cs:430 / void GiveGifts(MoveResult res)` | `Toast($"Gift: +1 {BoostName[pick]}!", null, 1.2f, 0.72f);` |
| `BlastBlocksGame.Hud.cs:76 / void Pill(float x, float y, float w, float h)` | `_levelText = HzUI.Label("Level", pill.transform, "", (int)(h * 0.55f), Color.white);` |
| `BlastBlocksGame.Hud.cs:95 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_goalTitle = HzUI.Label("GoalTitle", ct, "", 40, BbArt.Sub, TextAnchor.MiddleLeft);` |
| `BlastBlocksGame.Hud.cs:97 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_scoreText = HzUI.Label("Score", ct, "0", 76, BbArt.Ink, TextAnchor.MiddleLeft);` |
| `BlastBlocksGame.Hud.cs:99 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_subText = HzUI.Label("Sub", ct, "", 34, BbArt.Sub, TextAnchor.MiddleRight);` |
| `BlastBlocksGame.Hud.cs:104 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_comboText = HzUI.Label("Combo", _hudRoot, "", 54, BbArt.Gold);` |
| `BlastBlocksGame.Hud.cs:110 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_goalTitle = HzUI.Label("GoalTitle", ct, "", 40, BbArt.Sub);` |
| `BlastBlocksGame.Hud.cs:112 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_scoreText = HzUI.Label("Score", ct, "0", 96, BbArt.Ink);` |
| `BlastBlocksGame.Hud.cs:115 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_subText = HzUI.Label("Sub", ct, "", 34, BbArt.Sub);` |
| `BlastBlocksGame.Hud.cs:119 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `_comboText = HzUI.Label("Combo", _hudRoot, "", 50, BbArt.Gold);` |

| | |
|---|---|
| `BlastBlocksGame.Hud.cs:132 / void GoalCard(float x, float y, float w, float h, bool portrait)` | `var cnt = HzUI.Label("Count", slot, "0", portrait ? 52 : 56, BbArt.Ink, TextAnchor.MiddleLeft);` |
| `BlastBlocksGame.Hud.cs:156 / void BoosterButton(int b, float x, float y)` | `var btn = HzUI.Button("Booster" + b, _hudRoot, null, BoostColor[b], Color.white, () => OnBooster(bb), 40, (int)(s * 0.28f));` |
| `BlastBlocksGame.Hud.cs:172 / void BoosterButton(int b, float x, float y)` | `_boostBadge[b] = HzUI.Label("N", badgeBg.transform, "0", 38, Color.white); _boostBadge[b].rectTransform.Stretch();` |
| `BlastBlocksGame.Hud.cs:173 / void BoosterButton(int b, float x, float y)` | `var lbl = HzUI.Label("Name", _hudRoot, BoostName[b], 34, BbArt.Ink);` |
| `BlastBlocksGame.Hud.cs:183 / void BoosterButton(int b, float x, float y)` | `var lt = HzUI.Label("Lv", lk.transform, "Lv " + BoostUnlock[b], 36, Color.white);` |
| `BlastBlocksGame.Hud.cs:192 / void RefreshHud()` | `_levelText.text = _endless ? "Free Play" : "Level " + Level;` |
| `BlastBlocksGame.Hud.cs:198 / void RefreshHud()` | `_goalTitle.text = "SCORE";` |
| `BlastBlocksGame.Hud.cs:199 / void RefreshHud()` | `_subText.text = "Best " + Mathf.Max(Best, S.Score);` |
| `BlastBlocksGame.Hud.cs:204 / void RefreshHud()` | `_goalTitle.text = "GOAL " + D.target;` |
| `BlastBlocksGame.Hud.cs:205 / void RefreshHud()` | `_subText.text = IsPortrait ? "" : (D.par > 0 ? $"3 stars in {D.par} bricks" : "");` |
| `BlastBlocksGame.Hud.cs:210 / void RefreshHud()` | `_goalTitle.text = "COLLECT";` |
| `BlastBlocksGame.Hud.cs:211 / void RefreshHud()` | `_subText.text = "";` |
| `BlastBlocksGame.Hud.cs:222 / void RefreshHud()` | `_prizeCount[k].text = left > 0 ? left.ToString() : "";` |
| `BlastBlocksGame.Hud.cs:237 / void RefreshHud()` | `_comboText.text = S.Combo >= 2 ? "Combo x" + S.Combo : "";` |
| `BlastBlocksGame.Hud.cs:265 / void RefreshBoosters()` | `_boostBadge[b].text = ch > 0 ? ch.ToString() : "+";` |
| `BlastBlocksGame.Hud.cs:272 / void RefreshBoosters()` | `_boostLabel[b].text = !un ? "Locked" : b == BPocket && S != null && S.Pocket != null ? "Pocket" : BoostName[b];` |

| | |
|---|---|
| `BlastBlocksGame.Hud.cs:302 / void ShowLevelCard()` | `var p = Popup($"Level {Level}", msg, D.Goal == GoalKind.Collect ? BbArt.Pink : BbArt.Blue, ("Play", BbArt.Good, (Action)Begin));` |
| `BlastBlocksGame.Hud.cs:315 / void ShowLevelCard()` | `var nm = HzUI.Label("Name", p.Card, D.name, 38, BbArt.Sub);` |
| `BlastBlocksGame.Hud.cs:322 / void WinPopup(int stars)` | `var p = Popup("Level Complete!", " \n ", BbArt.Good, ("Next Level", BbArt.Blue, (Action)NextLevel), ("Replay", BbArt.Orange, (Action)StartLevel));` |
| `BlastBlocksGame.Hud.cs:324 / void WinPopup(int stars)` | `if (p.Message) { p.Message.text = $"\n\nScore {S.Score} Bricks {S.Pieces}"; p.Message.fontSize = 40; p.Message.rectTransform.anchoredPosition += new Vector2(0, -20); }` |
| `BlastBlocksGame.Hud.cs:341 / void UnlockPopup(int b, Action done)` | `var p = Popup("New Booster!", " \n \n ", BoostColor[b], ("Try it!", BbArt.Good, done));` |
| `BlastBlocksGame.Hud.cs:342 / void UnlockPopup(int b, Action done)` | `if (p.Message) { p.Message.text = $"\n\n{BoostName[b]}: {BoostDesc[b]}\nYou get {BoostFree[b]} free."; p.Message.fontSize = 38; p.Message.rectTransform.sizeDelta = new Vector2(740, 250); p.Message.rectTransform.anchoredPosition += new Vector2(0, -30); }` |
| `BlastBlocksGame.Hud.cs:355 / void ShowSettingsPopup()` | `var p = Popup("Settings", "", BbArt.Blue,` |
| `BlastBlocksGame.Hud.cs:356 / void ShowSettingsPopup()` | `(HzAudio.Muted ? "Sound: OFF" : "Sound: ON", BbArt.Blue, (Action)null),` |
| `BlastBlocksGame.Hud.cs:357 / void ShowSettingsPopup()` | `(HzAudio.MusicMuted ? "Music: OFF" : "Music: ON", BbArt.Blue, (Action)null),` |
| `BlastBlocksGame.Hud.cs:358 / void ShowSettingsPopup()` | `("How to Play", BbArt.Purple, (Action)(() => HelpPage(0))),` |
| `BlastBlocksGame.Hud.cs:364 / void ShowSettingsPopup()` | `sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(HzAudio.Muted ? "Sound: OFF" : "Sound: ON"); HzAudio.Play(Sfx.Toggle, 0.6f); });` |
| `BlastBlocksGame.Hud.cs:367 / void ShowSettingsPopup()` | `music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(HzAudio.MusicMuted ? "Music: OFF" : "Music: ON"); HzAudio.Play(Sfx.Toggle, 0.6f); });` |

| | |
|---|---|
| `BlastBlocksGame.Hud.cs:370 / void ShowSettingsPopup()` | `static readonly string[] HelpTitles = { "Place Bricks", "Clear Lines", "Goals", "Special Bricks", "Boosters" };` |
| `BlastBlocksGame.Hud.cs:384 / void HelpPage(int i)` | `(lastPage ? "Got it!" : "Next", BbArt.Good, (Action)(() => { if (!lastPage) HelpPage(i + 1); }));` |
| `BlastBlocksGame.Hud.cs:386 / void HelpPage(int i)` | `var pg = HzUI.Label("Page", p.Card, $"{i + 1} / {HelpPages.Length}", 32, BbArt.Sub);` |
| `BlastBlocksGame.Hud.cs:406 / void ShowTip(string text, float seconds)` | `var l = HzUI.Label("T", bg.transform, text, IsPortrait ? 42 : 38, Color.white, TextAnchor.MiddleCenter, false);` |
| `BlastBlocksGame.Hud.cs:551 / void CompleteGuide(int b)` | `Toast("Nice! You have " + Charges(b) + " " + BoostName[b] + (Charges(b) == 1 ? " charge" : " charges") + " left.", null, 1.8f, 0.78f);` |
| `BlastBlocksGame.View.cs:607 / void FloatText(string text, Vector2 at, Color color, int size, float dur)` | `var l = HzUI.Label("Float", _fxLayer, text, size, color);` |
| `BlastBlocksGame.View.cs:623 / void Praise(string text, bool big = false)` | `var l = HzUI.Label("Praise", _fxLayer, text, big ? 104 : 84, BbArt.Gold);` |

CHAPTER 11

# Artwork, fonts, audio and licences

Replace game images under Resources/blastblocks through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/blastblocks/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/blastblocks/Music/toybox_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

# Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

# Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

# Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.blastblocks`, version, scene orientation and signing. The build command writes `Builds/Android/blastblocks.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

# WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/blastblocks/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

# Troubleshooting

**Compile errors:** restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

**Edits do not appear:** stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

# Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.